

Reverse mode from local rules

Record the executed scalar graph, seed the loss, and propagate sensitivities backward.

THE QUESTION

How can one backward sweep give the loss's derivative with respect to every input and parameter?

THE INTUITION

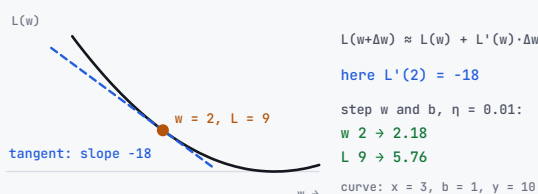
Break the computation into small operations that each know their own local slope. Walk the graph backwards from the loss: multiply what arrives by the local slope, and add up wherever paths meet.

KEY TAKEAWAY

Gradient sent back = gradient arriving $\times$ local slope, summed over paths. Backprop is the algorithm; autograd is the engine that records operations and runs their backward rules.

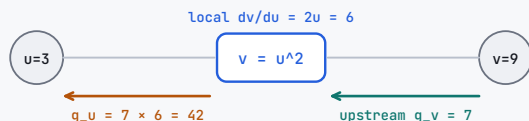
leaves $\rightarrow$ forward, store values $\rightarrow$ seed $g = 1$ at L $\rightarrow$ reverse sweep $\rightarrow$ accumulate $+=$ $\rightarrow$ read .grad, step

1 Derivative as a local model



Near u a scalar function is almost linear: $f(u + \Delta u) \approx f(u) + f'(u)\Delta u$. For many parameters, stack the partials into $\nabla_{\theta}\mathcal{L} = ((\partial\mathcal{L})/(\partial\theta_1), \dots)$. Gradient descent subtracts this vector. Reverse mode computes it efficiently when many inputs feed one scalar loss.

2 One operation, one reusable rule



For $v = f(u)$, if $g_v = \partial\mathcal{L}/\partial v$ arrives from downstream:

$$\underbrace{g_u}_{\text{sent to } u} = \underbrace{g_v}_{\text{arriving at } v} \cdot \underbrace{\frac{\partial v}{\partial u}}_{\text{local slope}}$$

Every operation needs only its local derivative and any forward values required to evaluate it.

3 Scalar rule table

add: copies

$v = a + b$
 $g_v = 3$
 $g_a = 3$
 $g_b = 3$

multiply: swaps

$v = a \cdot b, a=2, b=5$
 $g_v = 3$
 $g_a = 3 \cdot b = 15$
 $g_b = 3 \cdot a = 6$

ReLU off: blocks

$v = \text{ReLU}(u), u = -2$
 $g_v = 5$
 $g_u = 5 \cdot 0 = 0$

forward backward contributions

$v = a + b$ $g_a += g_v, g_b += g_v$

$v = ab$ $g_a += g_v b, g_b += g_v a$

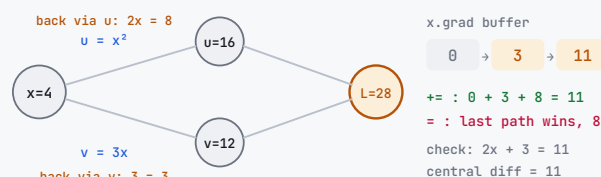
$v = u^2$ $g_u += g_v \cdot 2u$

$v = \varphi(u)$ $g_u += g_v \varphi'(u)$

$v = a - y$ $g_a += g_v, g_y -= g_v$

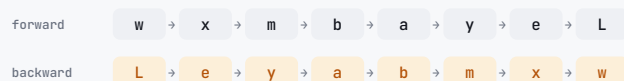
g_u is shorthand for $\partial\mathcal{L}/\partial u$; the subscript names the stored value. At exactly $u = 0$ ReLU has no derivative; software commonly uses 0.

4 Branches must accumulate



If x reaches the loss through children $u_1, \dots, u_k$: $g_x = \sum_{i=1}^k g_{u_i} (\partial u_i / \partial x)$. Use $+=$, not $=$: overwriting keeps one path and silently drops the rest. Example: $u = x^2, v = 3x, \mathcal{L} = u + v$ at $x = 4$ gives $g_x = 1(2x) + 1(3) = 8 + 3 = 11$.

5 Reverse-mode algorithm



- run the forward program and create value nodes
- store the operation and needed operands on each result
- topologically order the graph
- initialize every gradient to zero
- seed the scalar loss with $g_{\mathcal{L}} = 1$
- visit operations in reverse topological order; accumulate

6 Why reverse mode for training?

reverse mode $\bar{\bar{1}}$ forward + 1 backward

finite diff. 20,000,000 loss evals

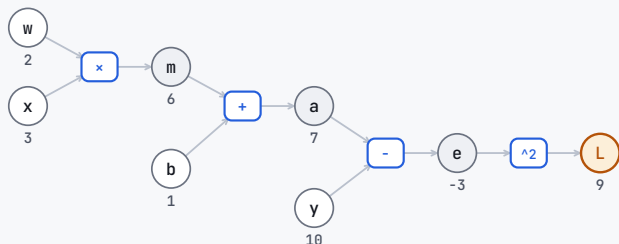
$P = 10^7$ parameters · bar length on a log scale

Training maps millions of parameters to one scalar loss. One reverse sweep returns every parameter's sensitivity at once. Central finite differences need two extra loss evaluations per checked coordinate: use them to debug, not to train.

Work one scalar graph end to end

Every stored value, local derivative and accumulated gradient, with numbers.

7 Model, loss, and primitive graph



white = leaves (given) · grey = stored intermediates · blue = operations

$\hat{y} = wx + b$, $\mathcal{L} = (\hat{y} - y)^2$ with $(w, x, b, y) = (2, 3, 1, 10)$.

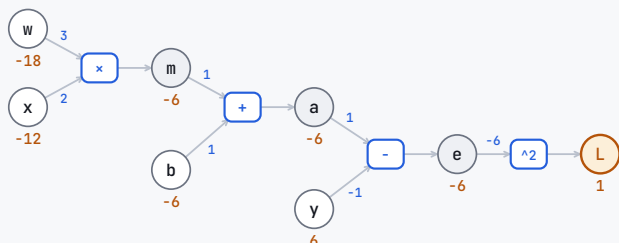
Name the intermediates:

$$m = wx = 6, \quad a = m + b = 7, \quad e = a - y = -3,$$

$$\mathcal{L} = e^2 = 9.$$

That decomposition is the graph: leaves (w, x, b, y) , stored values (m, a, e) , one scalar output. Save operands such as x, w, e : the multiply and square rules need them during backward.

8 Backward pass



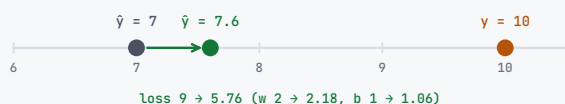
amber = $d\mathcal{L}/d(\text{node})$, seeded with 1 at L · blue = local slope on each edge
7 reverse edges; each multiplies upstream × local and adds (+=)

Seed $g_{\mathcal{L}} = 1$ and reverse the operations:

$$\begin{aligned} g_e &= 1 \cdot 2e = -6 & g_a &= g_e \cdot 1 = -6 \\ g_y &= (-6)(-1) = 6 & g_m &= g_b = g_a = -6 \\ g_w &= g_m x = -18 & g_x &= g_m w = -12 \end{aligned}$$

Final: $\partial \mathcal{L} / \partial (w, b, x, y) = (-18, -6, -12, 6)$.

9 Read the sign before updating



$\partial \mathcal{L} / \partial w = -18 < 0$, so a small descent step increases w : $w \leftarrow w - \eta(-18) = w + 18\eta$. With $\eta = 0.01$: $w : 2 \rightarrow 2.18$, $b : 1 \rightarrow 1.06$, and the prediction moves from 7 to 7.60, toward the target 10.

10 Autograd lifecycle in PyTorch

concept	PyTorch
leaf value	<code>requires_grad=True</code>
creator rule	<code>grad_fn</code> on a non-leaf
reverse sweep	<code>L.backward()</code>
leaf derivative	<code>p.grad</code>
keep an intermediate	<code>v.retain_grad()</code>
clear old gradients	<code>opt.zero_grad()</code>

```
w = torch.tensor(2., requires_grad=True)
b = torch.tensor(1., requires_grad=True)
x, y = torch.tensor(3.), torch.tensor(10.)
L = (w * x + b - y) ** 2
L.backward()
w.grad, b.grad # tensor(-18.), tensor(-6.)
```

Gradients accumulate by default: a second backward pass without clearing adds another contribution.

11 Independent gradient check

For one coordinate θ_j , the central difference

$$g_{\text{FD}} = \frac{\mathcal{L}(\theta_j + \varepsilon) - \mathcal{L}(\theta_j - \varepsilon)}{2\varepsilon}$$

should match autograd. Here it reproduces $(-18, -12, -6, 6)$ for (w, x, b, y) to within 4×10^{-10} . Compare in float64 at a few coordinates. Large disagreement usually means a graph, broadcasting, in-place mutation, nondifferentiability or numerical-scale bug.

12 From-scratch implementation audit

- ✓ Every result remembers its parents and local backward rule.
- ✓ Backward order is reverse topological, not recursive guesswork.
- ✓ Branches accumulate with +=.
- ✓ Only a scalar output is seeded automatically.
- ✓ Tests cover shared inputs and repeated use of one value.

13 Pause and think

Why seed with 1? $\partial \mathcal{L} / \partial \mathcal{L} = 1$; every other gradient is that 1 times local slopes, multiplied along paths and summed.

Why store forward values? Local slopes such as $2e$ and x need them; otherwise backward must recompute.

Does granularity matter? A sigmoid neuron as one fused node (8 edges) or as atomic ops (11 edges) gives the same $g_z = -0.25$.

Forward computes values · backward computes how the scalar loss responds to each stored value

Numbers from the Lecture 6 worked example, checked by hand rules, central differences and the executed PyTorch notebook. Slides and summary videos: nipunbatra.github.io/dl-2026.